

Claude Certified Associate – Foundations (CCAO-F) — Module 7 Practice Exam

Troubleshooting & Optimization

95 scenario-style and recall questions, multiple choice (A/B/C/D), one correct answer each.

Covers: Diagnosing Underperforming Prompts and Outputs · Adjusting Approach from Feedback and Results · Optimizing Workflows for Efficiency and Effectiveness.

Instructions: Select the single best answer for each question. An answer key with brief explanations is provided at the end of this document. Correct answers are distributed evenly across A, B, C and D. This practice material is based on user-provided course screenshots and is intended for self-study only — it is not an official Anthropic exam product.

A. Diagnosing Underperforming Prompts and Outputs

1. What are the two unproductive things most people do when an output disappoints?
 - A. Give up and conclude 'Claude can't do this,' or thrash by changing random things until something works
 - B. Switch to a higher model tier or restart the conversation immediately
 - C. Escalate to a colleague or lower the task's scope
 - D. Report a bug or request a refund
2. What does diagnostic skill replace those two defaults with?
 - A. A policy requiring escalation after one failed attempt
 - B. A single rule of thumb: always add more context
 - C. A sequence — underperformance has findable root causes, and running the sequence usually turns a dead end into a five-minute fix
 - D. A checklist that only applies to code-execution tasks
3. According to the module's first learning objective, what four root causes should underperformance be traced to?
 - A. Under-specification, wrong entry point, wrong model tier, or expired Memory
 - B. Under-specification, context overload, the wrong feature or model, or stale configuration
 - C. Prompt length, model tier, Skill trust, and data sensitivity
 - D. Context overload, wrong classification, missing gate, and bias
4. According to the second learning objective, what should recurring corrections turn into?
 - A. A permanent ban on the task type
 - B. A Memory entry that updates automatically
 - C. A justification for switching model tiers
 - D. Fixes that persist instead of repeating the same manual work
5. According to the third learning objective, what gets promoted into Projects, Skills, and standing instructions?
 - A. Memory entries and Incognito settings
 - B. Model tier selections and entry-point choices
 - C. Shared context, format, and verification steps
 - D. Delegation classifications and review gates

6. What is under-specification, and how is it characterized?
- A. A long conversation approaching its context limit, with earlier content auto-summarized
 - B. The prompt left out context, constraints, or the format — the most common cause, and the cheapest to fix: add what was missing
 - C. Asking for a calculation in prose instead of code execution
 - D. A standing instruction or Skill that has drifted out of date
7. What is context overload, and what does it show up as?
- A. A long conversation has approached the context limit, and earlier content has been automatically summarized to make room; detail can compress, so early instructions lose force
 - B. The prompt left out context, constraints, or the format
 - C. Asking for a deep analysis from a model tier chosen for speed
 - D. A knowledge source that has drifted out of date
8. What is the 'wrong feature or model' failure pattern?
- A. A standing instruction that has drifted out of date
 - B. The prompt left out context, constraints, or the format
 - C. A long conversation nearing its context limit
 - D. Asking for a calculation in prose instead of code execution, or a deep analysis from a model tier chosen for speed — the fix is the right tool, not more prompting
9. What is stale configuration, and where is its fix covered?
- A. A standing instruction, knowledge source, or Skill has drifted out of date and is degrading output silently; the fix is maintenance, covered in Module 5
 - B. The prompt left out context, constraints, or the format
 - C. A calculation requested in prose instead of code execution
 - D. A model tier chosen for speed instead of depth
10. Why does diagnosing the four failure patterns require reading the timing and pattern of the symptom, not just the bad output itself?
- A. Because only one of the four patterns actually occurs in practice
 - B. The four patterns produce similar-looking bad output, so the skill is reading the symptom for which one you are in
 - C. Because the four patterns always occur together
 - D. Because the output itself always names its own root cause
11. How does under-specification show up in a session?
- A. As 'it used to work,' when a setup that worked before suddenly does not
 - B. Partway through a good session, as the conversation grows
 - C. As a specific, repeatable error type on every run
 - D. From the very first response — the output was never right, because the prompt never carried what it needed
12. How does context overload show up in a session?
- A. As 'it used to work' a month later
 - B. From the very first response, before anything else has happened
 - C. Partway through a good session — it started fine and degraded as the conversation grew
 - D. As specific, repeatable numeric errors

- 13. How do wrong-feature-or-model failures show up?**
- A. From the very first response, with no prior successful use
 - B. As specific, repeatable error types — numbers that are subtly off, or shallow analysis on a task that needs depth
 - C. Partway through a long session, as context degrades
 - D. As a setup that worked last month and does not now
- 14. How does stale configuration show up?**
- A. As specific, repeatable numeric errors on every run
 - B. From the very first response, before anything else happens
 - C. Partway through a long session, as context degrades
 - D. As 'it used to work' — the same setup that produced good output last month now does not, because something it depends on drifted
- 15. Before blaming the tool, what four things should 'Isolating the cause' have you check?**
- A. The knowledge base, the standing instructions, the Memory, and the Skill
 - B. The model tier, the entry point, the Skill, and the connector
 - C. The prompt (specification), the context (window full or wrong material loaded), the feature choice (wrong entry point, model, or no code execution), or an expectation mismatch (the task was never one Claude could do well)
 - D. The reversibility, the consequence, the human element, and the accountability
- 16. What does naming which of the four causes it is do?**
- A. Points straight to the fix
 - B. Guarantees the output will improve on the next attempt
 - C. Determines which model tier to switch to
 - D. Confirms whether the task should be escalated to a human
- 17. What is step 1 of the diagnostic sequence?**
- A. Re-read the prompt against the five components: is anything under-specified?
 - B. Check the conversation length: is context overloaded and in need of a restart or summary?
 - C. Check the feature and model: is this a calculation that needs code execution?
 - D. Check the configuration: are the instructions, knowledge, and Skills current?
- 18. What is step 2 of the diagnostic sequence?**
- A. Check the feature and model: is this a calculation that needs code execution, or a complex task on a speed-tier model?
 - B. Re-read the prompt against the five components: is anything under-specified?
 - C. Check the conversation length: is context overloaded and in need of a restart or summary?
 - D. Check the configuration: are the instructions, knowledge, and Skills current?
- 19. What is step 3 of the diagnostic sequence?**
- A. Check the configuration: are the instructions, knowledge, and Skills current?
 - B. Re-read the prompt against the five components: is anything under-specified?
 - C. Check the conversation length: is context overloaded and in need of a restart or summary?
 - D. Check the feature and model: is this a calculation that needs code execution, or a complex task on a speed-tier model?

20. What is step 4 of the diagnostic sequence?
- A. Re-read the prompt against the five components: is anything under-specified?
 - B. Check the configuration: are the instructions, knowledge, and Skills current?
 - C. Check the conversation length: is context overloaded and in need of a restart or summary?
 - D. Check the feature and model: is this a calculation that needs code execution?
21. What is step 5 of the diagnostic sequence?
- A. Only then question whether the task is a fit at all
 - B. Immediately switch to the most capable model available
 - C. Escalate the task to a colleague
 - D. Restart the conversation from scratch
22. Why does the sequence run in this specific order?
- A. It runs from the most expensive fix to the cheapest, to rule out big problems first
 - B. It runs in alphabetical order of the four root causes
 - C. It runs in the order the four root causes were introduced earlier in the lesson
 - D. It runs cheapest-fix-first on purpose — re-reading the prompt costs seconds, while questioning whether the task fits at all is the most expensive conclusion and ends the attempt
23. What do most people invert, according to the lesson?
- A. They re-read the prompt first, when they should check configuration first
 - B. They jump to 'switch to the most capable model' or 'this task is impossible' first, which is the expensive move that usually was not needed
 - C. They check configuration first, when they should re-read the prompt first
 - D. They restart the conversation first, when they should check the model tier first
24. What is the discipline the sequence requires?
- A. Resisting the urge to run all five steps in parallel
 - B. Resisting the urge to skip step 1 and start with configuration
 - C. Resisting the urge to skip straight to step 5 (concluding the task is impossible) before running the cheaper steps first
 - D. Resisting the urge to document the diagnosis before fixing it
25. In the failure gallery, what was diagnosed for 'The summary keeps missing key points'?
- A. Context overload — restart from a summary
 - B. Under-specification — the prompt never said which points matter; fix: name the criteria for 'key'
 - C. Wrong feature — move the calculation to code execution
 - D. Stale configuration — run the Module 5 maintenance checklist
26. In the failure gallery, what was diagnosed for 'It stopped following my format halfway through'?
- A. Stale configuration — run the Module 5 maintenance checklist
 - B. Under-specification — name the criteria for 'key'
 - C. Wrong feature — move the calculation to code execution
 - D. Context overload in a long session — fix: restart from a summary, or persist the format to a standing instruction
27. In the failure gallery, what was diagnosed for 'The numbers are subtly wrong'?
- A. Wrong feature — fix: move the calculation to code execution
 - B. Under-specification — name the criteria that matter
 - C. Context overload — restart from a summary
 - D. Stale configuration — run the Module 5 maintenance checklist

28. In the failure gallery, what was diagnosed for 'It worked last month, now it's off'?
- A. Context overload — restart from a summary
 - B. Under-specification — name the criteria that matter
 - C. Stale configuration — fix: run the Module 5 maintenance checklist
 - D. Wrong feature — move the calculation to code execution
29. In the failure gallery, what was diagnosed for 'I asked it to predict next quarter's exact sales and the number was wrong'?
- A. Expectation mismatch — the task asks for something the tool cannot fix, predicting a future number with precision; the fix is to reshape the task, such as asking for a range with stated assumptions or a model of the drivers you can adjust
 - B. Under-specification — the prompt never named the target quarter
 - C. Wrong feature — the forecast should have used code execution alone
 - D. Stale configuration — the sales knowledge base had drifted out of date
30. What are the two places the diagnostic sequence usually ends, run end to end?
- A. A restart of the conversation, or a switch to Research mode
 - B. A model-tier upgrade, or an escalation to a human reviewer
 - C. A specific, cheap fix (most of the time), or a confident, reasoned 'this task needs reshaping' (occasionally)
 - D. A Skill rebuild, or a Project reconfiguration
31. What habit is the module building toward?
- A. Asking Claude to diagnose its own failure before responding again
 - B. Switching to a more capable model automatically the moment an output disappoints
 - C. Documenting every disappointing output for a monthly review
 - D. Reaching for the sequence automatically the moment an output disappoints, before forming an opinion about whose fault it is

B. Adjusting Approach from Feedback and Results

32. What is every disappointing output, according to this lesson?
- A. A sign the task should be reassigned to a human
 - B. Diagnostic data
 - C. A billing event that should be tracked
 - D. A signal to switch model tiers
33. What does letting that diagnostic data evaporate mean?
- A. Fixing the same problem by hand each time it recurs — a missed opportunity
 - B. Deleting the conversation after the task is complete
 - C. Failing to save the output as a file
 - D. Skipping the diagnostic sequence entirely
34. What is the skill this lesson teaches?
- A. Writing a standing instruction for every possible failure mode
 - B. Running the five-step diagnostic sequence before every prompt
 - C. Selecting the correct model tier for each recurring task
 - D. Translating output critique into a specific adjustment and then capturing the fix so it persists

35. For recurring work, what should each round's output be treated as?
- A. A final, unchangeable result
 - B. A signal about the setup that produced it
 - C. A billing consideration only
 - D. Evidence that the model tier should be upgraded
36. What does a recurring report needing the same manual correction every week tell you?
- A. The model tier is too low for the task
 - B. The task should be escalated to a different team
 - C. The prompt, context, or configuration is missing something
 - D. The report should be produced less frequently
37. What habit does the lesson recommend building after each round of recurring work?
- A. Asking whether the task should move to a new Project
 - B. Asking whether the model tier should be upgraded
 - C. Asking what the output revealed about the system that produced it
 - D. Asking whether the output should be discarded and restarted
38. Why does a vague 'this isn't quite right' not improve anything?
- A. It does not convert into a precise change — which component, which context, which configuration setting
 - B. Because Claude cannot process feedback phrased as a reaction
 - C. Because vague feedback always indicates a stale configuration
 - D. Because vague feedback should be escalated rather than acted on
39. How does 'too generic' translate into a specific adjustment?
- A. 'Move the calculation to code execution'
 - B. 'Add a tone constraint to the standing instruction'
 - C. 'Restart the session from a summary'
 - D. 'Add the audience to the context'
40. How does 'wrong tone' translate into a specific adjustment?
- A. 'Add the audience to the context'
 - B. 'Add a tone constraint to the standing instruction'
 - C. 'Name the criteria for “key”'
 - D. 'State the single question the output must answer up front'
41. What is the difference between a reaction and an instruction?
- A. A reaction names how the output feels; an instruction names what to change so the next output is different
 - B. A reaction is written by the user, while an instruction is written by Claude
 - C. A reaction applies to Chat, while an instruction applies only to Projects
 - D. A reaction is permanent, while an instruction is temporary
42. What is the reliable way to translate a reaction into an instruction?
- A. Ask Claude to rephrase the reaction as an instruction automatically
 - B. Ask what specifically would have to be present for this to be right, and which part of the setup controls that
 - C. Ask whether the reaction points to a stale configuration
 - D. Ask whether the task should be escalated instead of adjusted

43. How does 'missed the point' translate into a specific adjustment?
- A. 'Move the calculation to code execution'
 - B. 'Add the audience to the context'
 - C. 'Add a tone constraint to the standing instruction'
 - D. 'State the single question the output must answer up front'
44. What does each translation point at, and why does that matter?
- A. A specific entry point, which determines where the fix should be tested
 - B. A specific model tier, which determines how the fix should be applied
 - C. A specific lever — the prompt, the context loaded, or a configuration setting — and that lever is where the fix goes
 - D. A specific Delegation criterion, which determines who owns the fix
45. What happens if you cannot name the lever a critique points to?
- A. The critique should be discarded, since it cannot be acted on
 - B. The critique should be escalated to a human reviewer automatically
 - C. The critique should be treated as a stale-configuration issue by default
 - D. The critique is still a reaction, and the next attempt will be a guess
46. What should happen when a fix works, according to 'Capture what worked'?
- A. Do not leave it in a one-off conversation where it will be lost — promote it
 - B. Leave it in the conversation, since Memory will pick it up automatically
 - C. Report it to an administrator for organization-wide rollout
 - D. Discard it once the immediate task is complete
47. What does a phrasing that reliably produces the right output become, once promoted?
- A. A Memory entry
 - B. A Skill
 - C. A standing instruction
 - D. A knowledge base document
48. What does a multi-step fix become, once promoted?
- A. A standing instruction
 - B. A Skill
 - C. A Memory entry
 - D. A connector
49. What is the expensive failure mode this lesson names?
- A. Promoting a fix to a Skill when a standing instruction would have worked
 - B. Capturing a fix that later turns out to be wrong
 - C. Finding the fix and then losing it — a correction discovered in one session and not captured will be rediscovered later, at the same cost, multiplied by every person and every cycle
 - D. Relying on Claude's Memory instead of writing a standing instruction
50. What is the simple test for whether a fix is worth promoting?
- A. Will this same correction be needed again, by me or someone else? If yes, it belongs in configuration
 - B. Did the fix take more than five minutes to discover?
 - C. Did the fix require code execution to implement?
 - D. Did more than one person request the fix?

51. How does the lesson distinguish Claude's Memory from configuration, for the purpose of capturing a fix?
- A. Memory and configuration serve the same purpose and are interchangeable
 - B. Memory is shared across the team by default, while configuration is private to one user
 - C. Memory is the reliable home for a fix, while configuration is best-effort
 - D. Memory may pick up patterns you repeat, but it is per-user and best-effort; configuration is the deliberate, shared, reliable home for a fix
52. In the worked example, what two corrections did every campaign-brief draft need?
- A. It omitted the budget and buried the timeline
 - B. It omitted the target segment and buried the call to action
 - C. It used the wrong tone and the wrong format
 - D. It missed the audience and exceeded the word count
53. In that worked example, what did the marketer do instead of fixing both by hand each week?
- A. She switched to a more capable model for every brief
 - B. She built a Skill that rewrote every draft automatically
 - C. She added two standing instructions to the briefs Project: to always state the target segment in the first line, and to place the call to action in its own closing section
 - D. She asked Claude to remember the correction in Memory
54. What was the result after those standing instructions were added?
- A. The draft still needed manual correction, but took less time
 - B. The next draft arrived correct — one round of feedback, captured into configuration, removed a recurring weekly correction
 - C. The draft required an additional round of feedback before it was correct
 - D. The correction had to be reapplied the following week as well

C. Optimizing Workflows for Efficiency and Effectiveness

55. How is optimization described at the start of this lesson?
- A. Deliberate, not accidental — you instrument the workflow, find the friction, and promote the fix into configuration
 - B. Automatic once a Project has been set up correctly
 - C. A one-time setup task completed before a workflow's first run
 - D. Best left to whichever model tier is fastest
56. What does optimization compound into, when done well?
- A. A workflow that no longer needs a diagnostic sequence
 - B. A workflow that requires progressively less human oversight over time
 - C. A workflow that automatically selects the correct model tier
 - D. Each redundancy removed and each pattern promoted makes the workflow faster and more consistent for everyone who uses it
57. What should you look at in a recurring AI-assisted workflow to find redundancy and friction?
- A. The model tier assigned to each step of the workflow
 - B. The total number of conversations the workflow has used
 - C. The steps that repeat unnecessarily — the same context pasted every session, the same correction made every round, the same manual reformatting at the end
 - D. The number of people who have access to the workflow's Project

58. Why is friction easy to live with and hard to see?
- A. Because it only appears after a Project has been shared with a team
 - B. Because you absorb it one session at a time
 - C. Because it is hidden inside a Skill's bundled instructions
 - D. Because it only shows up in workflows that use code execution
59. What is the way to identify friction, according to the lesson?
- A. Review the Project's Memory entries for repeated corrections
 - B. Ask Claude to identify the friction in the workflow automatically
 - C. Compare the workflow's output against a competitor's process
 - D. Watch one full cycle of the workflow and note every step you do by hand that you also did last time
60. What should you do if you cannot see the friction on your own?
- A. Run the workflow once with the explicit job of writing down every manual step — that list is your optimization backlog
 - B. Assume there is no friction and move on
 - C. Escalate the workflow to a Developer or Architect for review
 - D. Switch to a higher model tier and observe whether the friction disappears
61. What is the fix for 'you paste or type the same thing every run'?
- A. Saved context or a standing instruction
 - B. A configuration change so the flaw stops appearing
 - C. A shared Skill or knowledge base so everyone runs the same setup
 - D. A restart of the conversation from a summary
62. What is the fix for 'you fix the same flaw in every output'?
- A. Switching to a more capable model tier
 - B. Saved context or a standing instruction
 - C. A shared Skill or knowledge base so everyone runs the same setup
 - D. A configuration change so the flaw stops appearing
63. What is the fix for 'different people running the same task get different results'?
- A. Saved context or a standing instruction
 - B. A shared Skill or knowledge base so everyone runs the same setup
 - C. A configuration change so the flaw stops appearing
 - D. A single person assigned to run the task going forward
64. What are the two moves that do most of the work in optimizing a workflow?
- A. Restart the conversation and re-read the five components
 - B. Upgrade the model tier and shorten the prompt
 - C. Consolidate steps that can run together rather than as separate prompts, and promote repeated patterns into Projects and Skills
 - D. Escalate the workflow and document the reasoning
65. Where does context that you paste every time belong?
- A. Claude's Memory
 - B. A Skill
 - C. The knowledge base or standing instructions
 - D. A connector

66. Where does a procedure you run every time belong?
- A. A standing instruction
 - B. A Skill
 - C. The knowledge base
 - D. Claude's Memory
67. What is promotion described as?
- A. How a personal workaround becomes a team-wide efficiency
 - B. How a Skill becomes eligible for Anthropic's built-in library
 - C. How a Project becomes eligible for Enterprise sharing
 - D. How a fix becomes exempt from further review
68. What is a standing instruction for, per the quick test in this lesson?
- A. A one-time correction that will not recur
 - B. Reference material every run needs
 - C. A repeatable procedure with steps
 - D. A rule that should always apply within a Project
69. What is the Project knowledge base for, per the quick test?
- A. A repeatable procedure with steps
 - B. A rule that should always apply within a Project
 - C. Reference material every run needs
 - D. A one-time correction that will not recur
70. What is a Skill for, per the quick test?
- A. A one-time correction that will not recur
 - B. A rule that should always apply within a Project
 - C. Reference material every run needs
 - D. A repeatable procedure with steps
71. What is the quick test for where a fix belongs?
- A. Is it reversible, consequential, or accountable? Score it against the Delegation criteria
 - B. Is it a rule, a reference, or a procedure? Rules go to instructions, references go to knowledge, procedures go to Skills
 - C. Is it urgent, routine, or exploratory? Route it by priority
 - D. Is it text, numeric, or visual? Route it by output type
72. Why does putting a fix in the wrong home cause some 'optimizations' to not stick?
- A. A procedure pasted as a one-line instruction loses its steps, and reference material crammed into an instruction bloats every prompt
 - B. Because the wrong home always triggers an error message
 - C. Because only Anthropic-provided Skills can hold procedures
 - D. Because standing instructions cannot reference the knowledge base
73. What caution does the lesson add about promoting a fix into configuration?
- A. Promote fixes only after they have been reviewed by an administrator
 - B. Promote every fix immediately, since delay compounds the cost of friction
 - C. Only promote fixes that have been used successfully at least ten times
 - D. Run the workflow a few cycles with the old approach still available before fully relying on the new setup, since not every change is an improvement

74. Why does the lesson recommend proving the gain on a few runs before committing?
- A. Because Claude requires several runs before a Skill can be finalized
 - B. The compounding benefit of optimization is real, but so is the compounding cost of a bad change baked into a workflow everyone uses
 - C. Because Projects cannot be edited more than once per day
 - D. Because standing instructions take several sessions to take effect
75. Why is optimization you cannot measure hard to justify or sustain?
- A. Because there is no concrete gain to track or point to
 - B. Because Claude will not apply an unmeasured optimization
 - C. Because unmeasured optimizations always turn out to be false improvements
 - D. Because measurement is required before a fix can be promoted to a Skill
76. What are the concrete gains the lesson says to track?
- A. Number of standing instructions written and knowledge base size
 - B. Model tier used, number of Skills enabled, and number of conversations held
 - C. Time saved per cycle, revision cycles reduced, and consistency improved across people doing the same task
 - D. Usage-limit consumption and account plan tier
77. What does identifying the right metric up front also tell you?
- A. Whether the workflow should be run in a Project or in Chat
 - B. Which model tier the workflow should use
 - C. When to stop — optimization has diminishing returns, and once the metric you care about is good enough, further tuning is its own kind of friction
 - D. How many people should be granted access to the workflow
78. In the worked example, what were the three frictions the audit found in the weekly reporting workflow?
- A. Each analyst uploads a different dataset, each uses a different format, and each writes different standing instructions
 - B. Each analyst uses a different model tier, each works in a separate Project, and each skips the review step
 - C. Each analyst re-types the same calculation, each uses a different entry point, and each has different Skill access
 - D. Each analyst re-pastes the same background, each reformats the output by hand, and each catches different things
79. In that same worked example, what was the optimization applied to each friction?
- A. The background moved into a shared Project knowledge base, a Skill was added for the report format, and a standing instruction was set for the verification step
 - B. The background moved into a Skill, the report format moved into the knowledge base, and the verification step moved into Memory
 - C. All three frictions were resolved by upgrading the model tier used by the team
 - D. All three frictions were resolved by consolidating the team into a single conversation
80. What was the measured result of that optimization?
- A. About 25 minutes per analyst, but format still varied by who ran it
 - B. About 25 minutes per analyst, consistent format across the team, and one fewer revision round
 - C. The same 45 minutes per analyst, but with fewer revision rounds
 - D. About 15 minutes per analyst, with no change in consistency

81. How does the worked example map onto the rule-reference-procedure test?
- A. The repeated background (a reference) went to the knowledge base, the report format (a procedure) went to a Skill, and the verification step (a rule) went to a standing instruction
 - B. The repeated background (a rule) went to a standing instruction, the report format (a reference) went to the knowledge base, and the verification step (a procedure) went to a Skill
 - C. All three frictions were treated as procedures and went to Skills
 - D. All three frictions were treated as rules and went to standing instructions

D. Quiz, Key Takeaways & Mixed Review

82. Quiz: You open a brand-new chat and ask Claude to draft a vendor renewal email. Its very first response is generic and ignores the two contract terms you most need to flag. Nothing degraded over time; the opening answer was already off. What is the most likely root cause?
- A. Stale configuration; a standing instruction has drifted out of date
 - B. Context overload; the conversation neared its limit and early instructions were summarized away
 - C. Under-specification; the prompt never named the contract terms or context that mattered, so the first answer was already off
 - D. Wrong feature; the task needed code execution
83. Quiz: You ask Claude for a thorough competitive analysis, but every response stays shallow and skims the surface even after you add detail. You had selected a fast, lightweight model tier to save time. Which diagnosis fits, and what is the fix?
- A. Stale configuration; the analysis depends on a knowledge source that has drifted
 - B. Under-specification; keep adding more context until the depth improves
 - C. Context overload; restart the session from a summary
 - D. Wrong model tier; switch to a more capable model built for deep analysis rather than a speed-tier model
84. Quiz: An output disappoints. What should you do before concluding Claude cannot do the task?
- A. Immediately switch to the most capable model
 - B. Run the diagnostic sequence: check specification, context length, feature/model, then configuration
 - C. Rewrite the prompt from scratch repeatedly until it works
 - D. Abandon the task and do it manually
85. Quiz: A recurring report needs the same manual correction every week. What is the most durable fix?
- A. Keep making the correction by hand each week
 - B. Capture the fix as a standing instruction or Skill in the Project so it persists
 - C. Switch models each week
 - D. Start a fresh conversation each time and re-explain
86. Quiz: A team's monthly close report takes about six hours and the numbers are reliable, but three analysts each format their section differently, so a reviewer spends about 90 minutes reconciling styles before sign-off. The goal is to cut that 90-minute reconciliation time. Which optimization delivers the biggest measured gain against that metric?
- A. Add a second reviewer to split the reconciliation work.
 - B. Upgrade the model tier the analysts use so each section is drafted faster.
 - C. Ask the analysts to start their sections earlier in the month.
 - D. Create one shared Skill that formats every section to a single template, so all three sections arrive in the same style.

87. According to Key Takeaway 1, what should happen before blaming the tool?
- A. Run the diagnostic sequence — specification, context, feature, configuration
 - B. Switch to a more capable model tier
 - C. Escalate the task to a human reviewer
 - D. Rewrite the prompt from scratch
88. According to Key Takeaway 2, what does isolating the failure require?
- A. Naming which entry point was used
 - B. Naming which model tier produced the output
 - C. Naming whether the failure is the prompt, the context, the feature choice, or an expectation mismatch
 - D. Naming which Delegation criterion applies
89. According to Key Takeaway 3, what should happen with a disappointing output?
- A. Escalate it to a colleague for a second opinion
 - B. Discard it and start over with a completely different prompt
 - C. Translate the critique into a specific adjustment, then capture the fix as an instruction or Skill so it persists
 - D. Log it and move on without further action
90. According to Key Takeaway 4, what does optimizing deliberately require?
- A. Upgrade the model tier used across the whole workflow
 - B. Instrument the workflow, find the friction, promote the fix into configuration, and measure the gain
 - C. Consolidate every workflow into a single shared Project
 - D. Escalate every recurring workflow to Developer expertise
91. Per the module's disclaimer, how should the course's examples and scenarios be treated?
- A. As unnecessary to verify against current documentation
 - B. As guaranteed accurate and unlikely to change
 - C. As legally binding professional guidance
 - D. As illustrative and often fictitious; mentioning a company or product does not imply endorsement in either direction
92. What does the module's sources note list as cross-references to earlier modules?
- A. Context management (Module 1), diagnostic iteration (Module 2), configuration maintenance (Module 5)
 - B. Prompt anatomy (Module 2), entry points (Module 1), Delegation mapping (Module 4)
 - C. Skill trust (Module 6), data sensitivity (Module 6), organizational policy (Module 6)
 - D. Requirements analysis (Module 4), research synthesis (Module 4), solution design (Module 4)
93. A team notices Claude's output has been degrading gradually over the course of a long single session, though the same setup worked fine at the start of the conversation. Applying this module's timing-pattern logic, which diagnosis fits best?
- A. Context overload — it started fine and degraded as the conversation grew
 - B. Under-specification — the prompt never carried what it needed from the start
 - C. Wrong feature or model — a calculation is being generated instead of computed
 - D. Stale configuration — a Skill or knowledge source has drifted out of date

94. A team wants both a fixable output problem addressed and the fix made durable for future use. Applying this module's throughline, what does that require in sequence?
- A. Rewrite the prompt from scratch, then repeat that rewrite at the start of every future session
 - B. Escalate the problem to a higher model tier, then document the escalation for future reference
 - C. Restart the conversation from a summary, then rely on Memory to retain the correction
 - D. Diagnose the failure with the sequence to find the specific fix, then capture that fix as a standing instruction or Skill so it does not have to be rediscovered
95. Which best summarizes the throughline connecting the three lessons of Module 7?
- A. Treat every disappointing output as an expectation mismatch and reshape the task rather than adjusting the prompt
 - B. Always switch to a more capable model whenever an output disappoints, since capability is the most common root cause
 - C. Diagnose underperformance with a disciplined, cheapest-first sequence, translate what the output reveals into a specific, capturable adjustment, and then optimize the whole workflow by finding friction and promoting fixes into the right configuration home
 - D. Optimize workflows first, then diagnose individual outputs only if the optimization does not resolve them

Answer Key & Explanations

1. **A** — The module opens by naming these two unproductive defaults before offering a sequence to replace them.
2. **C** — A disciplined sequence, not a single trick, is what the module builds toward.
3. **B** — These four root causes anchor the whole module.
4. **D** — Persistent fixes, not repeated manual corrections, is the stated goal.
5. **C** — Workflow optimization promotes these three things into configuration.
6. **B** — Under-specification is named as both the most common and the cheapest failure to fix.
7. **A** — Context overload is a session-length problem, distinct from a specification problem.
8. **D** — This pattern is a tool-selection problem, not a wording problem.
9. **A** — Stale configuration connects this module directly back to Module 5's maintenance lesson.
10. **B** — Similar-looking failures need a different diagnostic signal than just the output's content.
11. **D** — First-response-wrong is the signature timing of under-specification.
12. **C** — Degraded-over-time is the signature timing of context overload.
13. **B** — A specific, repeatable error type is the signature of a tool mismatch.
14. **D** — 'It used to work' is the signature timing of drifted configuration.
15. **C** — These four questions are the practical version of the four root causes.
16. **A** — Naming the cause is what makes the next step obvious rather than another guess.
17. **A** — The sequence starts with the cheapest possible check: re-reading the prompt.
18. **C** — Restarting from a summary costs a little more effort than re-reading, so it comes second.
19. **D** — Switching features or model costs more than the first two steps, so it comes third.
20. **B** — Checking configuration costs even more effort, so it is reserved for step 4.
21. **A** — Questioning fit is last because it is the most expensive conclusion — it ends the attempt.
22. **D** — Cost-ordering, cheapest first, is the explicit design principle behind the sequence.
23. **B** — This inversion — reaching for the expensive move first — is exactly what the ordered sequence corrects.
24. **C** — The discipline is specifically about not shortcutting to the expensive conclusion.
25. **B** — An unnamed criterion for 'key' is a classic under-specification gap.
26. **D** — Format compliance dropping partway through matches the context-overload timing signature.
27. **A** — Subtly wrong numbers are the signature of a prose calculation that should have run through code execution.
28. **C** — 'Worked last month, not now' is the exact phrasing the lesson attaches to stale configuration.
29. **A** — This is the step-5 case: no prompt, restart, feature, or configuration change fixes a request for precise prediction of an unknowable number.
30. **C** — These are the sequence's two legitimate endpoints.
31. **D** — The habit is reaching for the sequence first, before any conclusion about fault.
32. **B** — The lesson's opening line frames every disappointing output as data, not just a failure.
33. **A** — Evaporating data is the cost of not capturing a recurring fix.
34. **D** — This is the lesson's stated skill: translate, then capture.
35. **B** — Output is read as a signal about the system behind it, not just a one-off result.
36. **C** — A repeated correction points at a gap in the setup, not at the task itself.
37. **C** — This question after each round is how feedback loops get built into recurring work.
38. **A** — Vagueness fails specifically because it names no actionable lever.
39. **D** — This is the lesson's own example translation for 'too generic.'

- 40. **B** — This is the lesson's own example translation for 'wrong tone.'
- 41. **A** — This is the core distinction the lesson draws between feeling and fixing.
- 42. **B** — This question is the lesson's stated method for making the translation reliably.
- 43. **D** — This is the lesson's own example translation for 'missed the point.'
- 44. **C** — Naming the lever is what makes an adjustment actionable rather than aspirational.
- 45. **D** — An unnamed lever means the translation from reaction to instruction has not actually happened.
- 46. **A** — Promotion, not passive reliance on Memory, is what the lesson prescribes.
- 47. **C** — A single reliable phrasing is promoted to a standing instruction.
- 48. **B** — A multi-step fix is promoted to a Skill, matching the procedure-goes-to-Skill pattern.
- 49. **C** — Losing a found fix is the named failure mode, since it multiplies the same cost repeatedly.
- 50. **A** — Recurrence, not effort or popularity, is the deciding factor.
- 51. **D** — This distinction is why the lesson insists on deliberate promotion rather than relying on Memory.
- 52. **B** — These two recurring gaps are the ones the marketer noticed every week.
- 53. **C** — Two standing instructions, matched to two recurring rules, is the promotion she made.
- 54. **B** — One round of captured feedback eliminated the recurring weekly correction entirely.
- 55. **A** — Deliberate action, not automatic improvement, is the lesson's opening premise.
- 56. **D** — The compounding benefit is speed and consistency across everyone who uses the workflow.
- 57. **C** — Repeated manual steps are exactly what the friction audit looks for.
- 58. **B** — Friction accumulates invisibly because each individual instance feels minor in the moment.
- 59. **D** — A single observed cycle, with manual steps noted, is the lesson's method.
- 60. **A** — A deliberate step-logging run is the fallback when friction is not obvious.
- 61. **A** — This is the first of the three named friction signals and its matching fix.
- 62. **D** — This is the second named friction signal and its matching fix.
- 63. **B** — This is the third named friction signal and its matching fix.
- 64. **C** — Consolidate and promote are named as the two moves that do most of the optimization work.
- 65. **C** — Repeated context is reference or rule material, which belongs in knowledge or instructions.
- 66. **B** — A repeatable multi-step procedure is exactly what a Skill is for.
- 67. **A** — Promotion turns an individual's workaround into something the whole team benefits from.
- 68. **D** — Rules go to instructions, per the quick test.
- 69. **C** — References go to knowledge, per the quick test.
- 70. **D** — Procedures go to Skills, per the quick test.
- 71. **B** — This three-way test is the lesson's method for choosing the right configuration home.
- 72. **A** — Mismatched homes degrade the fix itself, which is why the promotion has to land correctly.
- 73. **D** — A trial period before full reliance is the lesson's stated caution.
- 74. **B** — Both the benefit and the risk of a shared workflow change compound, which is why the change is proven first.
- 75. **A** — Without a tracked gain, there is nothing to point to that justifies keeping the change.
- 76. **C** — These three concrete gains are what the lesson recommends tracking.
- 77. **C** — Knowing the target metric is also what tells you when further optimization stops paying off.
- 78. **D** — These three frictions were what the audit surfaced in the 45-minute workflow.
- 79. **A** — This maps each friction to its correct configuration home, matching the rule-reference-procedure test.
- 80. **B** — Time, consistency, and revision rounds all improved together in the worked example.
- 81. **A** — Each friction was sorted correctly by type: reference, procedure, and rule, into their matching homes.

82. **C** — First-response-wrong, in a brand-new chat, is the signature of under-specification, not the other three patterns.
83. **D** — Adding detail did not help, which rules out under-specification; a speed-tier model on a depth task is the named wrong-feature-or-model case.
84. **B** — The full ordered sequence runs before any conclusion that the task itself is unfit.
85. **B** — Only capturing the fix into configuration removes the recurring manual cost.
86. **D** — The bottleneck is inconsistent formatting, not drafting speed or reviewer capacity, so a shared formatting Skill targets the actual metric.
87. **A** — Takeaway 1: underperformance has discoverable causes.
88. **C** — Takeaway 2: isolate before you fix.
89. **C** — Takeaway 3: every disappointing output is data.
90. **B** — Takeaway 4: optimize deliberately.
91. **D** — The disclaimer explicitly frames examples as illustrative and non-endorsing, matching earlier modules.
92. **A** — These three specific cross-references are named in the sources section.
93. **A** — Fine-at-the-start, degraded-over-time is the timing signature the module attaches specifically to context overload.
94. **D** — Diagnosis finds the fix; capture is what makes the fix durable — the two lessons work in sequence.
95. **C** — Each lesson supplies one stage of the same practice: diagnose the specific failure, translate and capture the fix, then optimize the workflow it belongs to.